

Week 2 Day 4 Hands-on Lab: Runtime Config and Observability

이 lab은 Compose를 사용하지 않는다. 같은 image라도 실행 옵션이 달라지면 상태가 달라진다는 것을 docker run, logs, inspect, exec, stats, failure drill로 확인한다.

Phase A: preflight

```
cd /mnt/d/paperclip
docker version
docker image ls nginx
docker image ls postgres
```

Expected:

Docker client/server 정보가 출력된다.
nginx 또는 postgres image가 없으면 이후 run 단계에서 pull된다.

Phase B: env file 준비

```
cd /mnt/d/paperclip
mkdir -p week2/day4/labs/env-report
sed -n '1,120p' week2/day4/labs/env-report/.env.example
cp week2/day4/labs/env-report/.env.example week2/day4/labs/env-report/.env
sed -n '1,120p' week2/day4/labs/env-report/report.sh
```

Expected:

```
APP_ENV=practice
FEATURE_FLAG=on
DB_PASSWORD=change-me-locally
```

주의: DB_PASSWORD 값은 실습용 placeholder다. 실제 개인 password, cloud key, Docker Hub token을 넣지 않는다.

.env는 다음처럼 한 줄에 하나씩 기록한다.

```
APP_ENV=practice
FEATURE_FLAG=on
DB_PASSWORD=change-me-locally
# DEBUG=true
```

env file은 container 실행 시점에 다음처럼 로드한다.

```
docker run --rm --env-file week2/day4/labs/env-report/.env alpine:3.20 env
```

파일 형식은 다음 규칙을 따른다.

형식	의미
KEY=value	container 안에 KEY=value로 주입
KEY	host shell에 export된 같은 이름의 값을 사용
# comment	comment로 무시

이미 실행된 container는 env file을 다시 읽지 않는다. .env를 고쳤다면 container를 새로 만들어야 한다.

환경별 파일을 나누는 패턴도 확인한다.

```
cat > week2/day4/labs/env-report/.env.dev <<'EOF'
APP_ENV=dev
FEATURE_FLAG=on
API_BASE_URL=http://localhost:3000
EOF

cat > week2/day4/labs/env-report/.env.staging <<'EOF'
APP_ENV=staging
FEATURE_FLAG=on
API_BASE_URL=https://staging.example.com
EOF

cat > week2/day4/labs/env-report/.env.prod <<'EOF'
APP_ENV=prod
FEATURE_FLAG=off
API_BASE_URL=https://api.example.com
EOF

docker run --rm --env-file week2/day4/labs/env-report/.env.dev alpine:3.20 sh
-c 'echo "$APP_ENV $FEATURE_FLAG $API_BASE_URL"'
```

```
docker run --rm --env-file week2/day4/labs/env-report/.env.staging alpine:3.20
sh -c 'echo "$APP_ENV $FEATURE_FLAG $API_BASE_URL"'
docker run --rm --env-file week2/day4/labs/env-report/.env.prod alpine:3.20 sh
-c 'echo "$APP_ENV $FEATURE_FLAG $API_BASE_URL"'
```

Expected:

```
dev on http://localhost:3000
staging on https://staging.example.com
prod off https://api.example.com
```

해석: image는 같고 env file만 바꿔 환경 차이를 표현한다. 이 패턴은 나중에 Kubernetes의 ConfigMap/Secret, Terraform의 dev.tfvars, staging.tfvars, prod.tfvars로 이어진다.

Phase C: -e와 --env-file 비교

```
docker run --rm -e APP_ENV=inline -e FEATURE_FLAG=off alpine:3.20 env | sort |
grep -E 'APP_ENV|FEATURE_FLAG'
docker run --rm --env-file week2/day4/labs/env-report/.env alpine:3.20 env |
sort | grep -E 'APP_ENV|FEATURE_FLAG|DB_PASSWORD'
docker run --rm --env-file week2/day4/labs/env-report/.env alpine:3.20 sh -c
'echo "APP_ENV=$APP_ENV FEATURE_FLAG=$FEATURE_FLAG DB_PASSWORD=$DB_PASSWORD"'
```

Expected:

```
APP_ENV=inline
FEATURE_FLAG=off
APP_ENV=practice
FEATURE_FLAG=on
DB_PASSWORD=change-me-locally
APP_ENV=practice FEATURE_FLAG=on DB_PASSWORD=change-me-locally
```

해석: -e는 명령줄에 직접 들어가고, --env-file은 파일에서 읽는다. 둘 다 runtime config이며 image를 새로 build하지 않는다.

비교	-e	--env-file
입력 위치	실행 명령 안	별도 파일
적합한 경우	한두 개 값을 임시로 바꿀 때	여러 값을 재사용할 때
기록 위험	shell history에 값이 직접 남기 쉬움	파일 관리가 필요함
공유 방식	명령 예시만 공유	.env.example만 공유
적용 시점	container 생성 시점	container 생성 시점

실제 기록에는 password 값을 그대로 남기지 않는다.

```
DB_PASSWORD=***masked***
```

Phase D: nginx run/logs/HTTP

```
docker rm -f paperclip-day4-nginx || true
docker run -d --name paperclip-day4-nginx -p 18084:80 nginx:1.27-alpine
docker ps --filter name=paperclip-day4-nginx
docker logs paperclip-day4-nginx --tail 30
curl -I http://localhost:18084
```

Expected:

```
STATUS Up
0.0.0.0:18084->80/tcp
HTTP/1.1 200 OK
```

해석: Up은 process 상태이고, HTTP/1.1 200 OK는 서비스 관점의 확인이다.

환경 설정을 로그로 남기는 경우에는 secret masking을 확인한다.

```
docker rm -f paperclip-day4-log-env || true
docker run --name paperclip-day4-log-env --env-file week2/day4/labs/env-report/.env -v "$PWD/week2/day4/labs/env-report:/work:ro" alpine:3.20 /work/report.sh || true
docker logs paperclip-day4-log-env
```

Expected:

```
APP_ENV=practice
FEATURE_FLAG=on
DB_PASSWORD=***masked***
```

Phase E: inspect와 exec

```
docker inspect paperclip-day4-nginx --format 'Ports={{json
.NetworkSettings.Ports}}'
docker inspect paperclip-day4-nginx --format 'Image={{.Config.Image}} Restart=
{{json .HostConfig.RestartPolicy}}'
docker exec paperclip-day4-nginx ls -l /usr/share/nginx/html
docker exec paperclip-day4-nginx sh -c 'ps | head'
docker rm -f paperclip-day4-env-inspect || true
docker run -d --name paperclip-day4-env-inspect --env-file
week2/day4/labs/env-report/.env alpine:3.20 sleep 300
docker inspect paperclip-day4-env-inspect --format '{{range .Config.Env}}
{{println .}}{{end}}' | grep -E 'APP_ENV|FEATURE_FLAG|DB_PASSWORD'
docker exec paperclip-day4-env-inspect sh -c 'env | grep -E
"APP_ENV|FEATURE_FLAG|DB_PASSWORD" '
```

Expected:

```
Ports={"80/tcp":[{"HostIp":"0.0.0.0","HostPort":"18084"}]}
Image=nginx:1.27-alpine
index.html
nginx
APP_ENV=practice
FEATURE_FLAG=on
DB_PASSWORD=change-me-locally
```

해석: inspect/exec는 env 값을 보여줄 수 있으므로 제출물에는 DB_PASSWORD=***masked***로 남긴다.

Phase F: stats와 restart policy

```
docker stats paperclip-day4-nginx --no-stream
docker inspect paperclip-day4-nginx --format 'before={{json
.HostConfig.RestartPolicy}}'
docker update --restart unless-stopped paperclip-day4-nginx
docker inspect paperclip-day4-nginx --format 'after={{json
.HostConfig.RestartPolicy}}'
docker rm -f paperclip-day4-restart-missing-env || true
docker run -d --name paperclip-day4-restart-missing-env --restart on-failure:2
postgres:16-alpine
```

```
sleep 3
docker inspect paperclip-day4-restart-missing-env --format 'RestartCount=
{{.RestartCount}} Status={{.State.Status}} ExitCode={{.State.ExitCode}}'
docker logs paperclip-day4-restart-missing-env --tail 20 || true
```

Expected:

```
NAME                CPU %      MEM USAGE / LIMIT
before={"Name":"no","MaximumRetryCount":0}
after={"Name":"unless-stopped","MaximumRetryCount":0}
RestartCount=2
POSTGRES_PASSWORD
```

해석: restart policy는 process를 다시 띄우려는 정책이지 원인을 고치는 기능이 아니다.

Phase G: failure drill 1 - missing env

```
docker rm -f paperclip-day4-pg-missing-env || true
docker run --name paperclip-day4-pg-missing-env postgres:16-alpine || true
docker ps -a --filter name=paperclip-day4-pg-missing-env
docker logs paperclip-day4-pg-missing-env --tail 40 || true
```

Expected failure output:

```
Error: Database is uninitialized and superuser password is not specified.
You must specify POSTGRES_PASSWORD to a non-empty value
STATUS Exited
```

Hint:

postgres image는 최초 DB 초기화에 POSTGRES_PASSWORD가 필요하다.
container가 Exited라면 docker logs가 첫 확인 명령이다.

Fix and recheck:

```
docker rm -f paperclip-day4-pg-missing-env || true
docker run -d --name paperclip-day4-pg-ok -e POSTGRES_PASSWORD=practice-only
postgres:16-alpine
docker logs paperclip-day4-pg-ok --tail 40
```

Expected recovery:

```
database system is ready to accept connections
```

Wrong env file path:

```
docker run --rm --env-file week2/day4/labs/env-report/.env.production
alpine:3.20 env || true
ls week2/day4/labs/env-report
```

Expected failure output:

```
no such file or directory
```

Hint:

실행 전에 env file 경로가 틀린 문제다. app이나 image 문제가 아니다.

Phase H: failure drill 2 - wrong port

```
curl -I http://localhost:80 || true
curl -I http://localhost:18084 || true
docker ps --filter name=paperclip-day4-nginx
```

Expected failure output:

```
curl: (7) Failed to connect to localhost port 80
HTTP/1.1 200 OK
0.0.0.0:18084->80/tcp
```

Hint:

host는 18084로 접근한다. container 내부 80은 host 80을 자동으로 열지 않는다.

Phase I: failure drill 3 - wrong network

```
docker rm -f paperclip-day4-net-web || true
docker network rm paperclip-day4-net-a paperclip-day4-net-b || true
docker network create paperclip-day4-net-a
docker network create paperclip-day4-net-b
docker run -d --name paperclip-day4-net-web --network paperclip-day4-net-a
nginx:1.27-alpine
```

```
docker run --rm --network paperclip-day4-net-b alpine:3.20 wget -S -O-  
http://paperclip-day4-net-web || true
```

Expected failure output:

```
bad address 'paperclip-day4-net-web'
```

Fix and recheck:

```
docker run --rm --network paperclip-day4-net-a alpine:3.20 wget -S -O-  
http://paperclip-day4-net-web | head
```

Expected recovery:

```
HTTP/1.1 200 OK
```

Phase J: failure drill 4 - stale volume

```
docker rm -f paperclip-day4-pg-volume || true  
docker volume rm paperclip-day4-pgdata || true  
docker run -d --name paperclip-day4-pg-volume -e POSTGRES_PASSWORD=practice-  
only -e POSTGRES_DB=first -v paperclip-day4-pgdata:/var/lib/postgresql/data  
postgres:16-alpine  
sleep 5  
docker rm -f paperclip-day4-pg-volume  
docker run -d --name paperclip-day4-pg-volume -e POSTGRES_PASSWORD=practice-  
only -e POSTGRES_DB=second -v paperclip-day4-pgdata:/var/lib/postgresql/data  
postgres:16-alpine  
docker logs paperclip-day4-pg-volume --tail 30  
docker volume inspect paperclip-day4-pgdata
```

Expected signal:

```
Database directory appears to contain a database; Skipping initialization
```

Hint:

env를 바꿔도 기존 volume data가 남아 있으면 최초 초기화가 다시 실행되지 않는다.
volume 삭제는 data reset이므로 의도적으로 판단해야 한다.

Phase K: cleanup/security audit

```
docker ps -a --filter name=paperclip-day4
docker network ls | grep paperclip-day4 || true
docker volume ls | grep paperclip-day4 || true
docker system df
```

Cleanup:

```
docker rm -f paperclip-day4-nginx paperclip-day4-log-env paperclip-day4-env-
inspect paperclip-day4-pg-ok paperclip-day4-pg-missing-env paperclip-day4-net-
web paperclip-day4-restart-missing-env paperclip-day4-pg-volume || true
docker network rm paperclip-day4-net-a paperclip-day4-net-b || true
rm -f week2/day4/labs/env-report/.env week2/day4/labs/env-report/.env.dev
week2/day4/labs/env-report/.env.staging week2/day4/labs/env-report/.env.prod
# data reset이 필요할 때만 실행
# docker volume rm paperclip-day4-pgdata
```

Phase L: Compose mapping preview

지금 실행한 option	Day 5에서 옮길 Compose 항목
-p 18084:80	services.web.ports
-e POSTGRES_PASSWORD=...	services.db.environment
--env-file .env	env_file 또는 \${VARIABLE}
.env.dev/.env.staging/.env.prod	환경별 env file
--network paperclip-day4-net-a	networks
docker logs <name>	docker compose logs <service>
docker exec <name> ...	docker compose exec <service> ...